CI/CD Pipeline for Containerized Web Application Using Jenkins & Docker

❖ Introduction to Project

This project demonstrates a fully automated CI/CD pipeline using Jenkins Declarative Pipeline. The pipeline pulls source code from GitHub, builds a Docker image, pushes the image to Docker Hub, and deploys the application on a Docker-enabled Jenkins agent.

A multi-node Jenkins setup is used, where the Jenkins master orchestrates the pipeline and the worker node performs Docker operations such as building, pushing, and running containers.

This setup reflects real-world DevOps practices and provides hands-on experience with CI/CD automation, Docker image management, and Jenkins Pipeline as Code.

❖ Objectives

The main objectives of this project are:

- To implement a **Declarative Jenkins Pipeline** for automated CI/CD.
- To build a Docker image using a project's Dockerfile directly on the Jenkins agent.
- To securely authenticate and push the Docker image to Docker Hub using Jenkins credentials.
- To deploy the updated container on a Jenkins agent by stopping/removing the old container and starting a new one.
- To establish and utilize a **multi-node Jenkins setup**, where the build and deployment run on a worker node with Docker installed.
- To store the pipeline logic in a Jenkinsfile inside the GitHub repository (Pipeline-as-Code best practice).

❖ Prerequisites

To run this CI/CD pipeline, the following prerequisites must be fulfilled:

1. Jenkins Requirements

- Jenkins installed and running
- Pipeline plugin enabled
- Git plugin installed
- One Jenkins agent (worker node) connected
- Agent must have Docker installed
- Agent must have a label (e.g., slave-1 or worker_node)

2. Repository Requirements

- GitHub repo containing:
  - Dockerfile
  - Application files (index.html or similar)
  - Jenkinsfile (Declarative Pipeline)

3. DockerHub Requirements

- Docker Hub account
- Personal Access Token (PAT)
- Jenkins credentials created with ID (e.g., docker_hub_cred)

Step 1: Configure Docker Access for Jenkins User

To allow Jenkins to run Docker commands during the pipeline execution, the Jenkins user must be added to the Docker group. This removes the need for using sudo while executing Docker operations.

Commands executed:

usermod -aG docker jenkins

systemctl restart Jenkins

```
root@ip-172-31-21-178:~# usermod -aG docker jenkins
root@ip-172-31-21-178:~# systemctl restart jenkins
root@ip-172-31-21-178:~#
```

Verified that Docker is installed and running correctly on the master node.

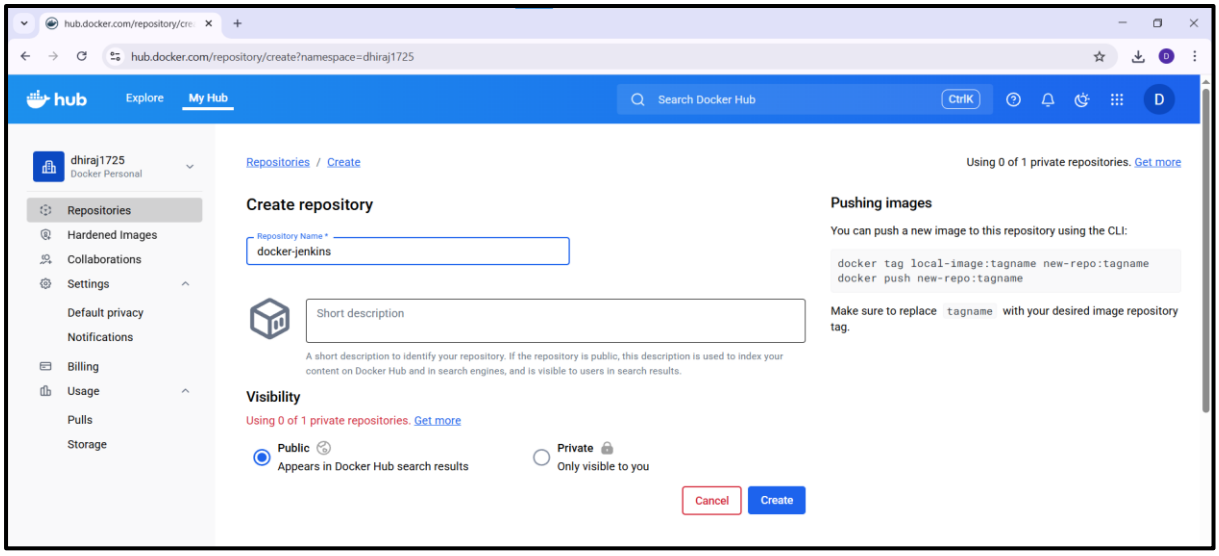
```
root@ip-172-31-21-178:~# docker version
Client: Docker Engine - Community
 Version:           29.1.2
 API version:       1.52
 Go version:        go1.25.5
 Git commit:        890dcca
 Built:             Tue Dec  2 21:55:19 2025
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:          29.1.2
  API version:      1.52 (minimum version 1.44)
  Go version:       go1.25.5
  Git commit:       de45c2a
  Built:            Tue Dec  2 21:55:19 2025
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.2.0
  GitCommit:        1c4457e00facac03ce1d75f7b6777a7a851e5c41
 runc:
  Version:          1.3.4
  GitCommit:        v1.3.4-0-gd6d73eb8
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
root@ip-172-31-21-178:~#
```

Step 2: Create Docker Hub Repository, Generate Access Token & Add Jenkins Credentials

To enable Jenkins to push Docker images to Docker Hub, a repository and secure authentication token must be created, then added as credentials in Jenkins.

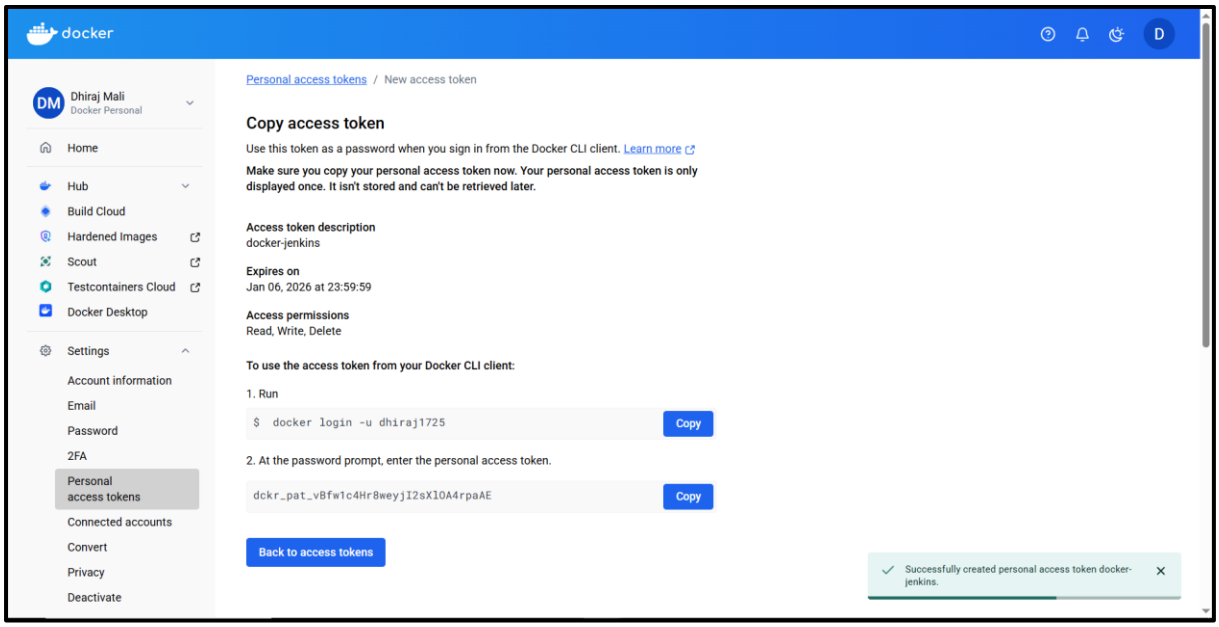
a) Created Docker Hub Repository

- Repository Name: docker-jenkins
- Visibility: Public



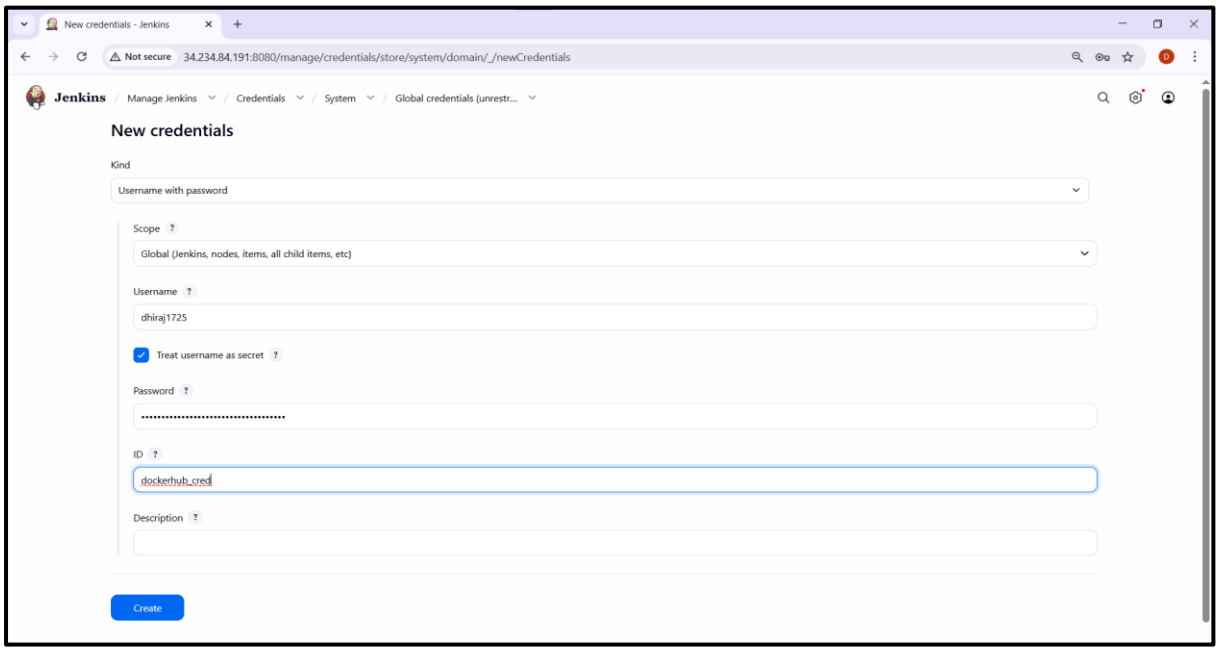
b) Generated Access Token

- Token Permission: Read, Write, Delete
- Token is used in place of Docker password during Jenkins pipeline execution.



c) Added Docker Credentials in Jenkins

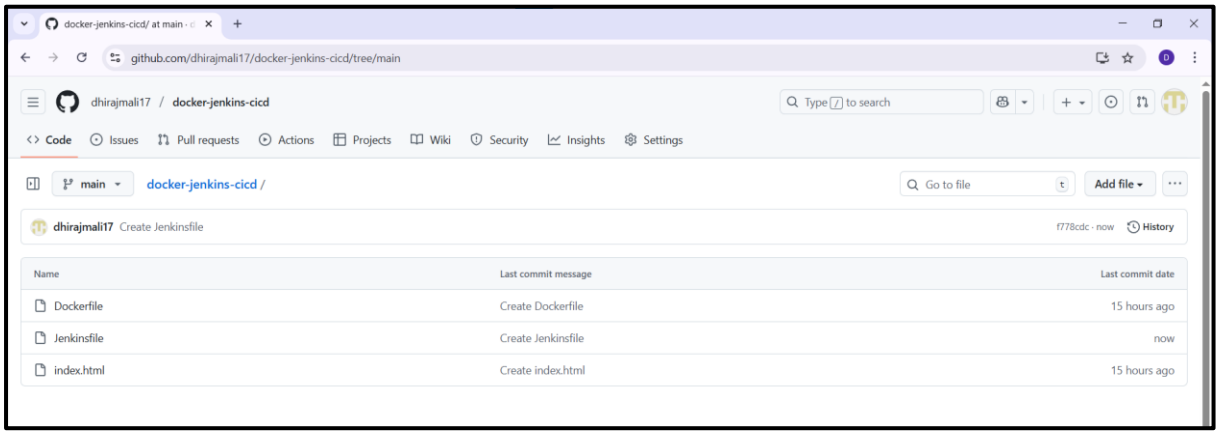
- Kind: *Username with Password*
- Username: Docker Hub username
- Password: PAT (Access Token)
- Credential ID: dockerhub_cred



- Jenkins now has secure access to Docker Hub.
- The pipeline can authenticate, push images, and manage Docker repositories automatically.

Step 3: Create GitHub Repository and Add Required Project Files

A GitHub repository was created to store all the necessary files for the CI/CD pipeline. This repository acts as the source for Jenkins to fetch code, build Docker images, and deploy the container.



Files Added to the Repository

a) Dockerfile

Defines how the Docker image is built using the Apache HTTPD base image.

```
FROM httpd:2.4

# Copy your website files to Apache document root

COPY ./index.html /usr/local/apache2/htdocs/

# Expose port 80

EXPOSE 80
```

b) index.html

A simple webpage served by the deployed Docker container.

```
<!DOCTYPE html>
<html>
<head>
    <title>My Jenkins CI/CD Web App</title>
</head>
<body>
    <h1>Welcome to My Dockerized CI/CD Pipeline!</h1>
    <p>This page is served by an Apache HTTPD container.</p>
</body>
</html>
```

c) Jenkinsfile

Defines the Declarative Jenkins Pipeline to automate build, push, and deployment steps.

```
pipeline {
  agent { label 'slave-1' }


  environment {
    DOCKERHUB_USER = 'dhiraj1725'
    IMAGE_REPO     = 'docker-jenkins'
    IMAGE_TAG      = 'latest'
    CONTAINER_NAME = 'myhttpd'
    HOST_PORT      = '8080'
    CONTAINER_PORT = '80'
```

```
}

triggers {
  pollSCM('* * * * *')
}

stages {
  stage('Checkout') {
    steps { checkout scm }
  }

  stage('Build image') {
    steps {
      sh '''
        echo "Building Docker image..."
        docker build -t ${DOCKERHUB_USER}/${IMAGE_REPO}:${IMAGE_TAG} .
      '''
    }
  }

  stage('Login to DockerHub') {
    steps {
      withCredentials([usernamePassword(credentialsId: 'dockerhub_cred',
                                        usernameVariable: 'DH_USER',
                                        passwordVariable: 'DH_PASS')]) {
```

```
        sh 'echo "$DH_PASS" | docker login -u "$DH_USER" --password-stdin'
      }
    }
  }

  stage('Push image') {
    steps {
      sh '''
        echo "Pushing image to Docker Hub..."
        docker push ${DOCKERHUB_USER}/${IMAGE_REPO}:${IMAGE_TAG}
      '''
    }
  }

  stage('Deploy container') {
    steps {
      sh '''
        set -e
        if docker ps -aq -f name=${CONTAINER_NAME} | grep -q . ; then
          echo "Stopping and removing old container..."
          docker rm -f ${CONTAINER_NAME} || true
        fi
        echo "Starting container ${CONTAINER_NAME}..."
        docker run -d --name ${CONTAINER_NAME} -p
${HOST_PORT}:${CONTAINER_PORT}
${DOCKERHUB_USER}/${IMAGE_REPO}:${IMAGE_TAG}
```

```
            echo "Website available at http://<your-server-ip>:${HOST_PORT}"
        '''
      }
    }
  }

  post {
    always {
      sh 'docker logout || true'
    }
  }
}
```

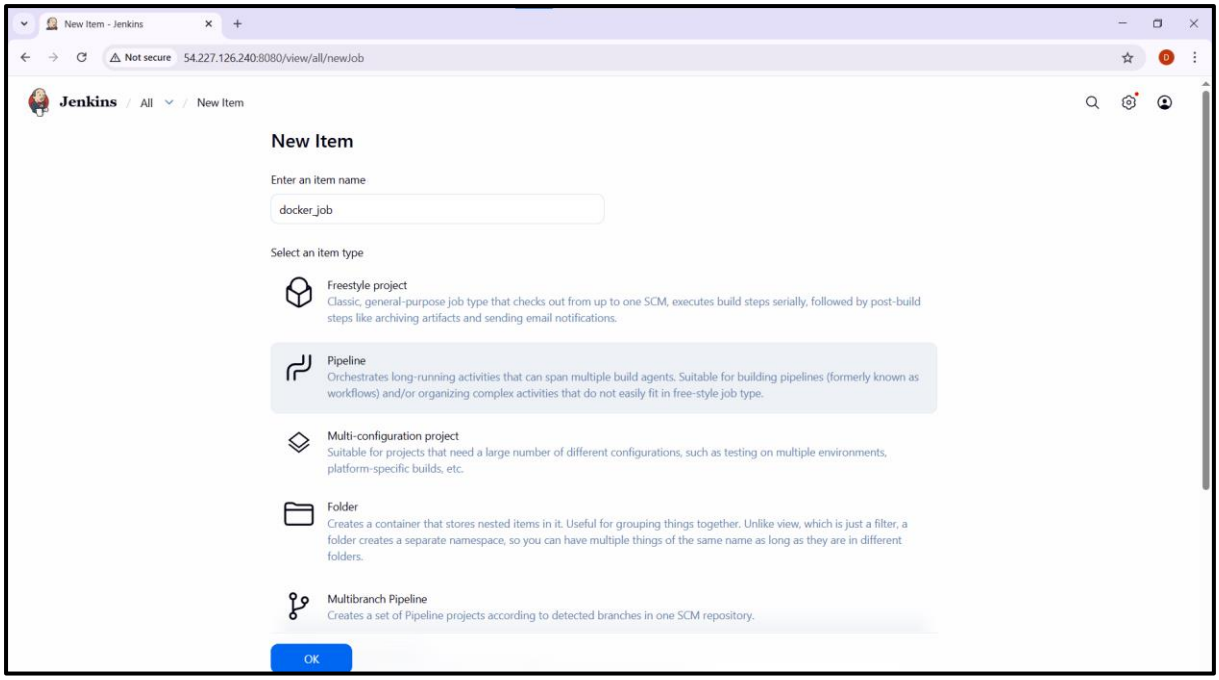
- Repository successfully structured with Dockerfile, HTML file, and Jenkinsfile.
- Jenkins can now clone this repo and run the complete CI/CD pipeline automatically.

Step 4: Create Jenkins Pipeline Job and Configure SCM Integration

A new Jenkins Pipeline job was created to automatically fetch the GitHub repository and execute the Jenkinsfile stored inside it.

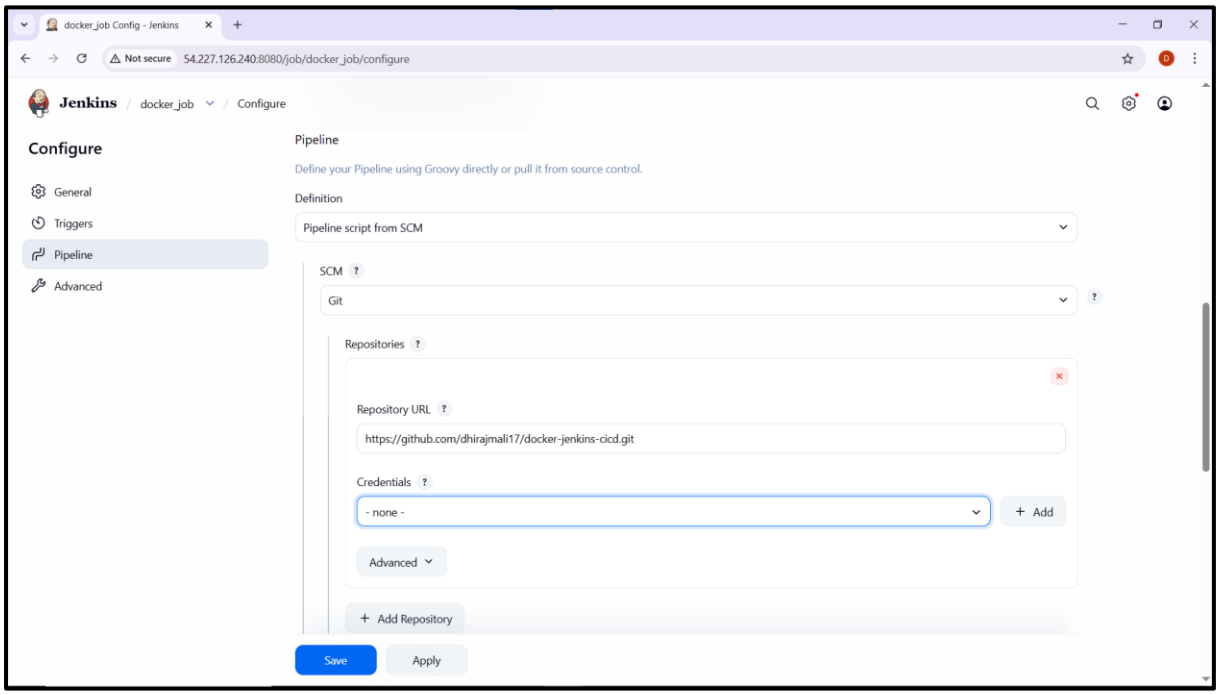
a) Created a New Jenkins Job

- Job Name: **docker_job**
- Selected **Pipeline** as the project type.

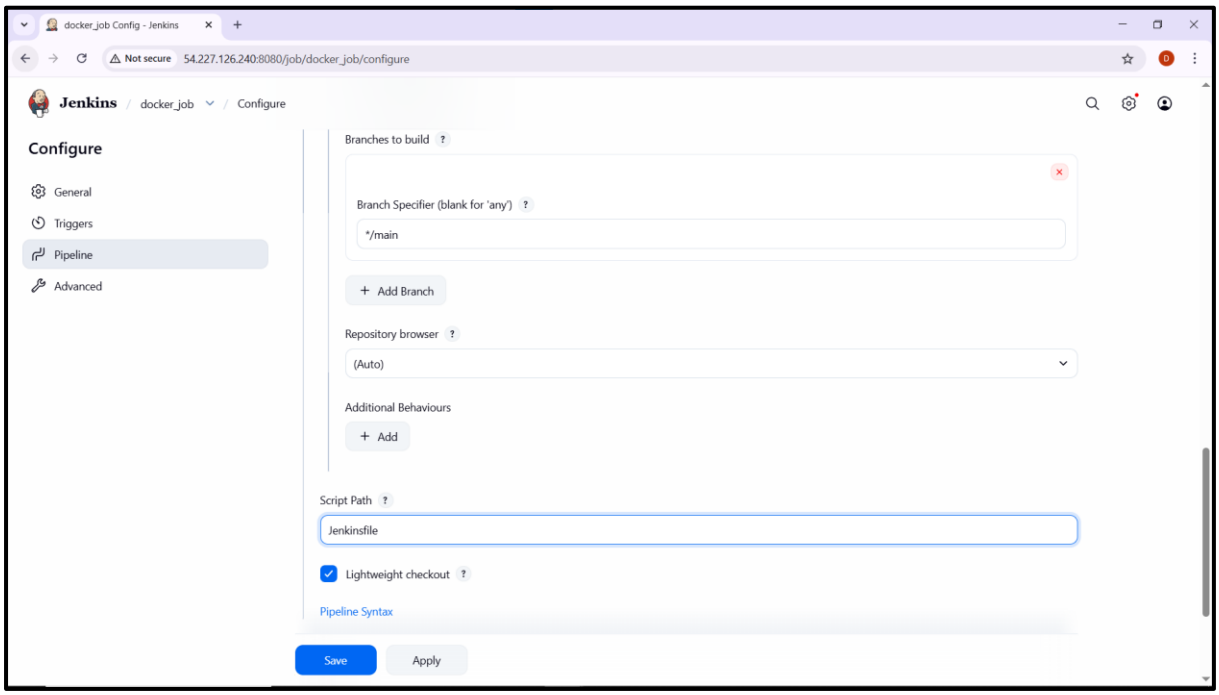


b) Configured Pipeline to Use GitHub Repository

- Definition: **Pipeline script from SCM**
- SCM: **Git**
- Repository URL: https://github.com/dhirajmali17/docker-jenkins-cicd.git



- Branch to build: */main
- Script Path: Jenkinsfile



c) Saved the Configuration and Triggered the First Build

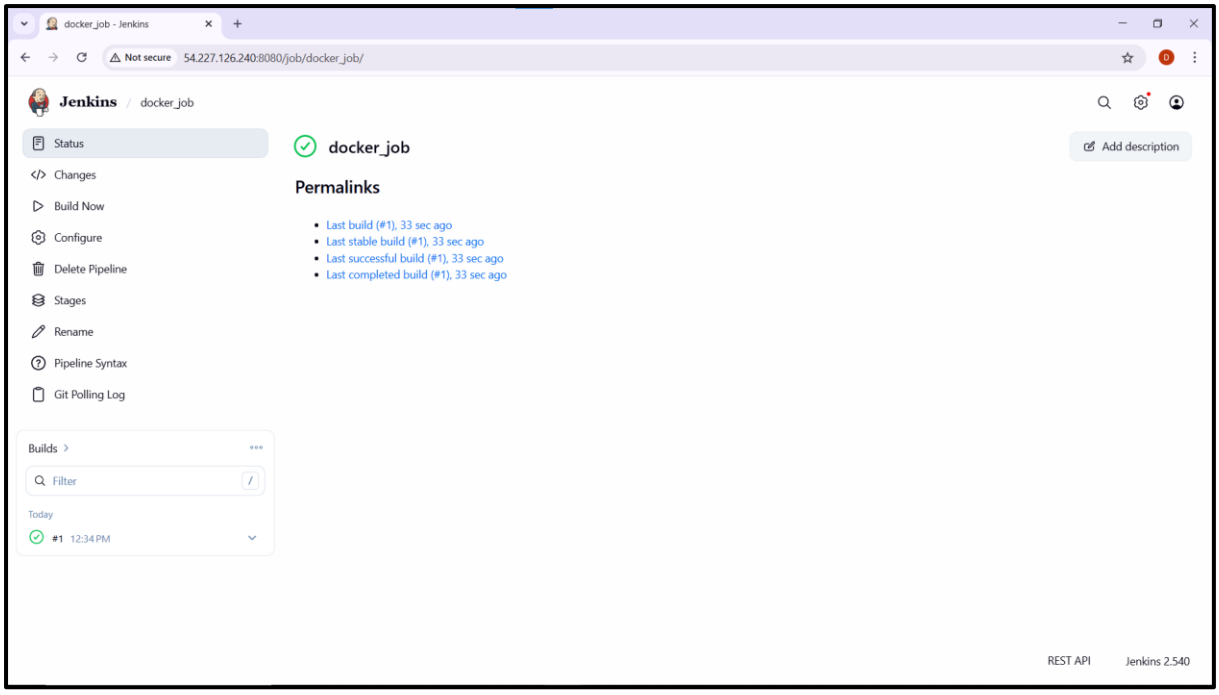
Jenkins automatically pulled the repo, read the Jenkinsfile, and executed all pipeline stages.

Step 5: Run Pipeline & Verify Deployment

Execute the Jenkins pipeline and verify that the image is built, pushed to Docker Hub, and the container is deployed on the worker node. Capture pipeline output and validate the running web app.

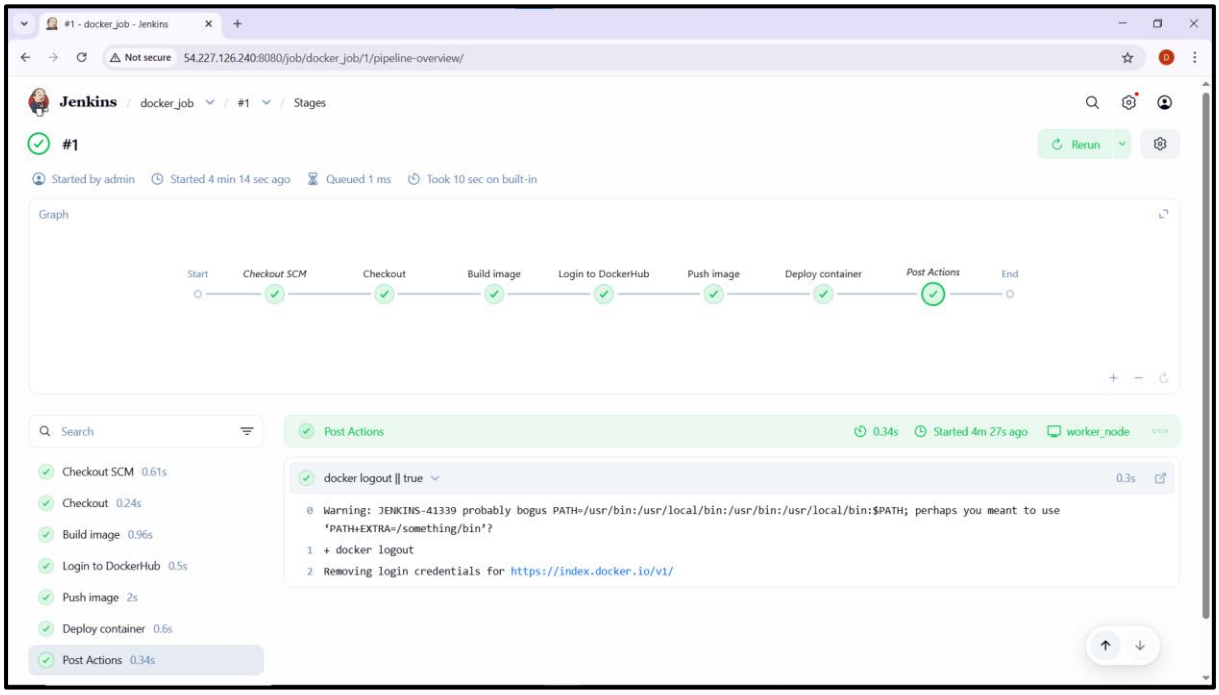
a) Trigger the pipeline

- In Jenkins → open the pipeline job (docker_job) → click **Build Now** or run from the pipeline page.
- Expected: a new build appears in the Builds panel.



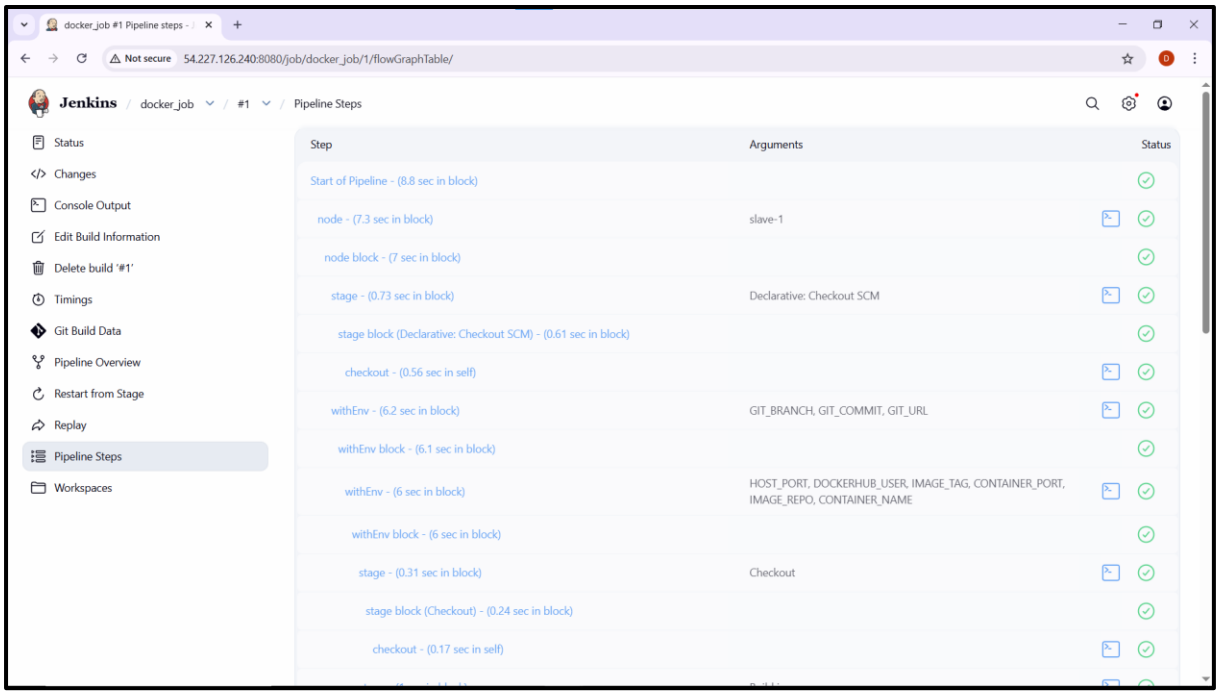
b) Open Pipeline overview

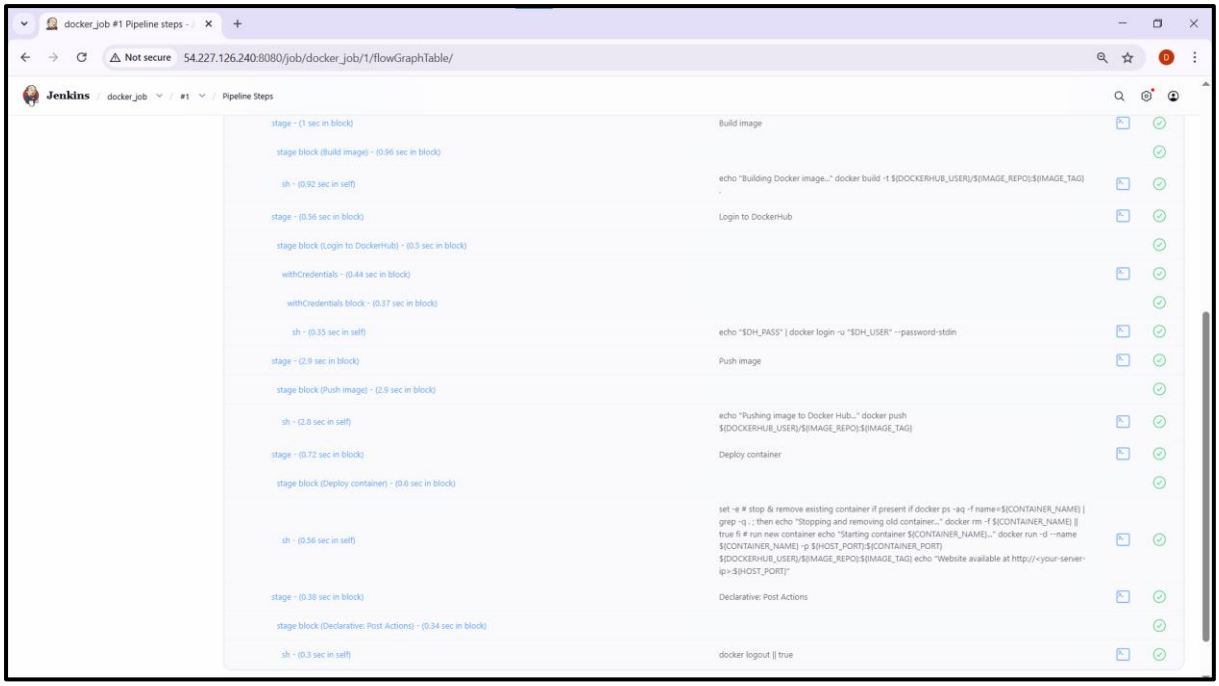
- Click the latest build → **Pipeline Steps / Pipeline Overview**.
- Verify each stage (Checkout SCM → Checkout → Build image → Login to DockerHub → Push image → Deploy container → Post Actions) completes with green check marks.



c) View Detailed Pipeline Steps

- Jenkins executed each block inside the Jenkinsfile.
- You can see:
  - Docker build command
  - Docker login
  - Docker push
  - Old container removal
  - New container deployment
- Every step has a green ✓ meaning no failure.





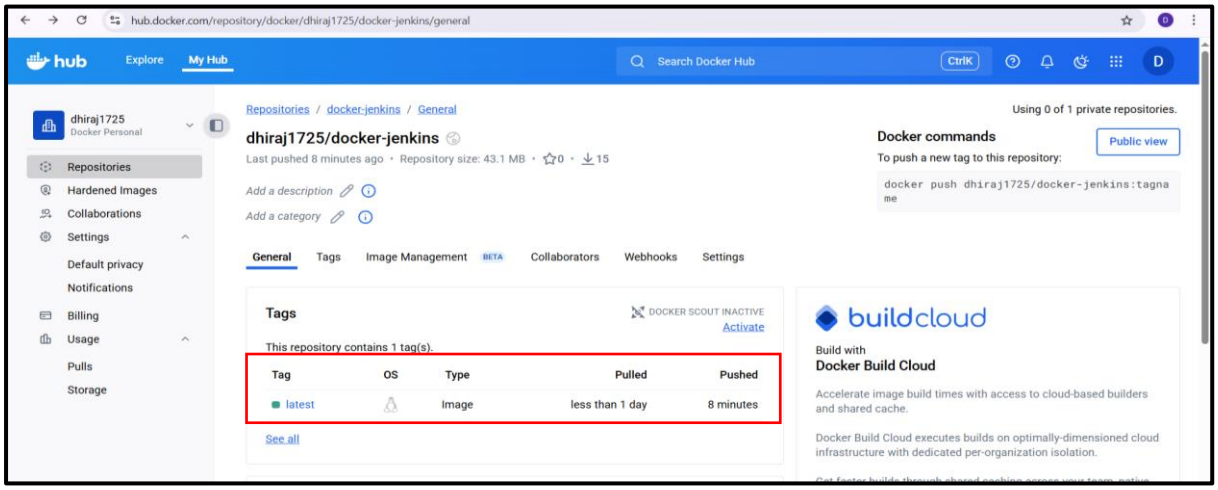
Step 6: Verify Image Push and Application Deployment

In this final step, I verified that the CI/CD pipeline successfully pushed the Docker image to Docker Hub and deployed the updated container on the worker node.

a) Verify Docker Image on Docker Hub

After the pipeline completed, I logged in to Docker Hub and confirmed that the new image tag **latest** was successfully pushed.

- Repository: **dhiraj1725/docker-jenkins**
- Tag: **latest**
- Last pushed: a few minutes ago



b) Verify Running Container on Worker Node

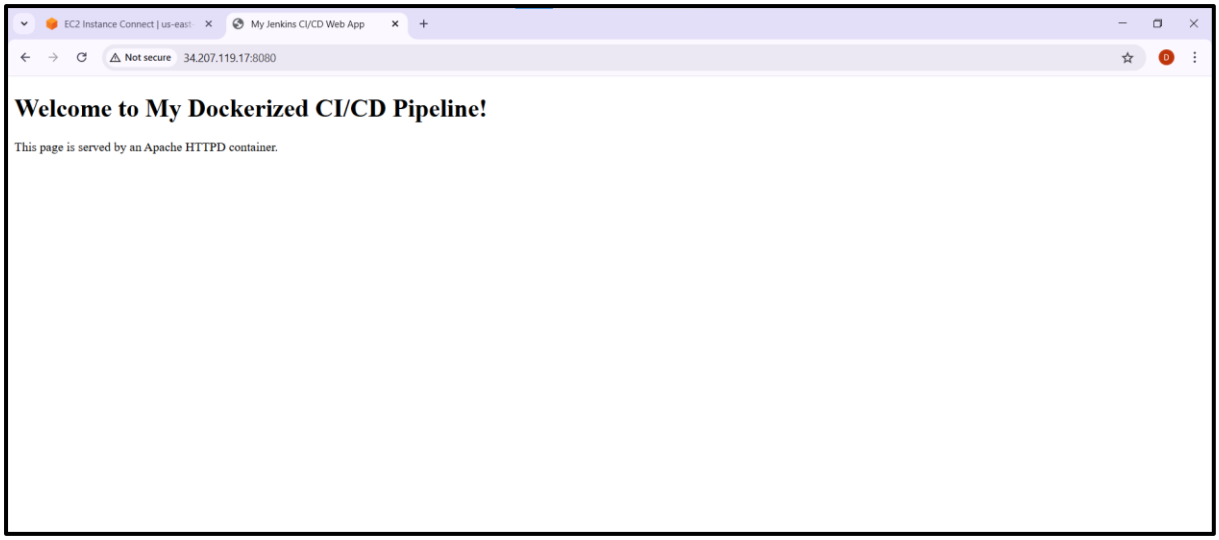
- Using EC2 Instance Connect, I checked the active containers: **docker ps**
- The container **myhttpd** was running correctly, mapped to port **8080** → **80**.
  - Image used: dhiraj1725/docker-jenkins:latest
  - Status: **Up**
  - Port Mapping: 0.0.0.0:8080 -> 80/tcp

c) Access the Web Application in Browser

- Finally, I accessed the deployed application using:

  http://<EC2-Public-IP>:8080

- The Apache HTTPD container successfully served the index.html page created in the repository.
  - Website loaded successfully
  - Confirms end-to-end CI/CD pipeline functionality



❖ Conclusion

This project successfully implemented a fully automated **Dockerized CI/CD Pipeline using Jenkins Pipeline and multi-node architecture**. By integrating GitHub, Jenkins Master, Jenkins Worker Node, Docker Hub, and container deployment, the entire build–push–deploy workflow was automated without any manual intervention.

The pipeline performed the following operations end-to-end:

- Pulled the latest source code and Dockerfile from GitHub
- Built a new Docker image on the worker node
- Logged in to Docker Hub using secure credentials
- Pushed the updated image to a private Docker Hub repository
- Automatically deployed the latest container on the worker node
- Served the web application successfully through an Apache HTTPD container

This project demonstrates how CI/CD can streamline application delivery, reduce deployment errors, and ensure consistent and reliable releases. It also highlights the importance of automation in DevOps and shows how Jenkins Pipelines offer better scalability and version control compared to freestyle jobs.

Overall, the project achieved its objective of creating a robust, automated, and production-ready CI/CD pipeline for containerized applications.